

KTH Technology and Health

Objektorienterad programmering

Grundläggande begrepp

Varför ytterligare en kurs i programmering?





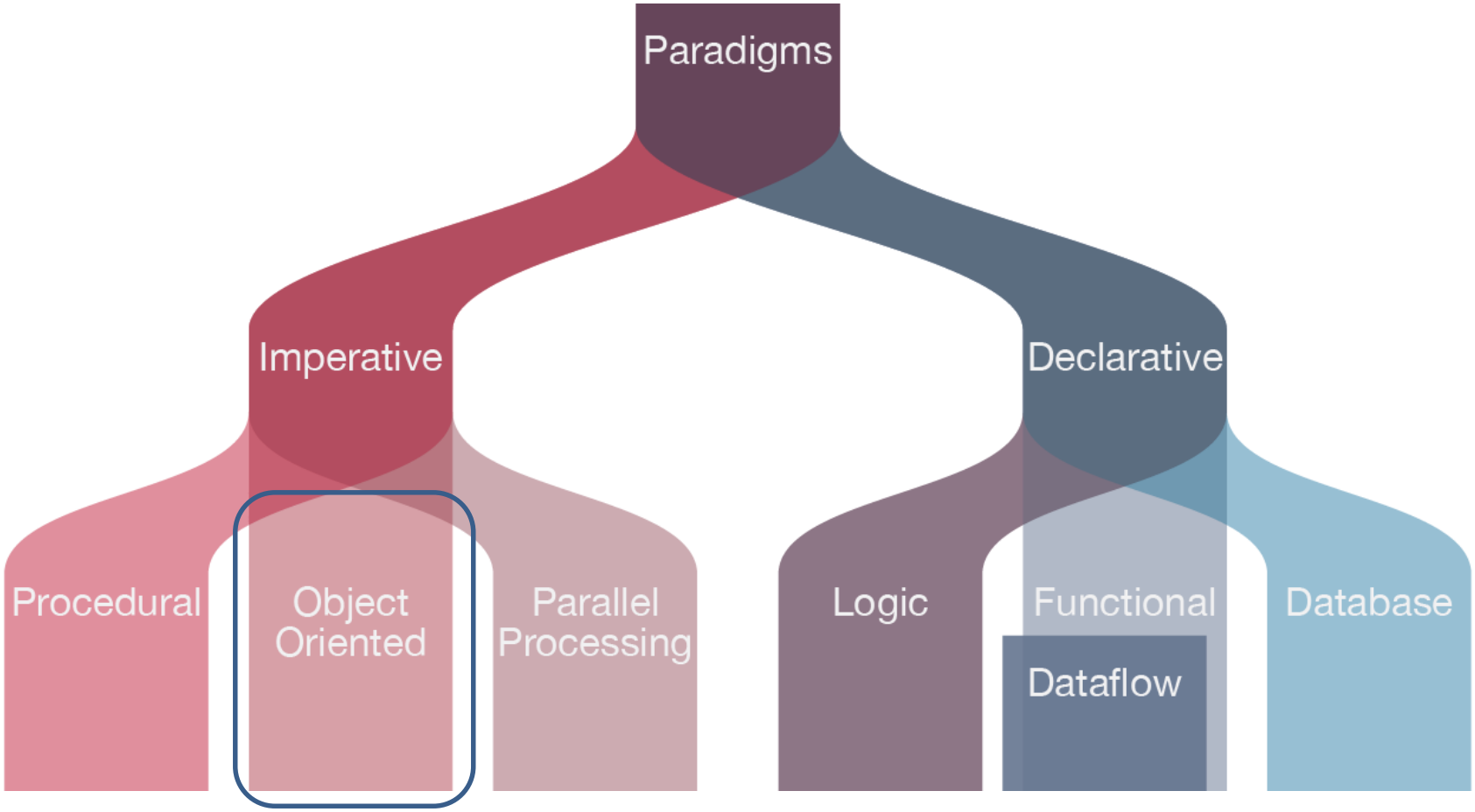




Varför ytterligare en kurs i programmering?

- För större, komplicerade, applikationer och system behöver vi kunna skriva en strukturerad modell av *vad* som ska modelleras i applikationen
  - Modell (här) ≠ flödesschema
- Abstrahera från verklighetens objekt till objekt i applikationen
  - Modellera lösningen, i form av objekt, före algoritmen
  - Fokus på vad som ska modelleras/abstraheras => verklighetsnära
- Exempel
  - System för bank: Användare, Kund, Konto, Valuta, Transaktion, …
  - Kortspel: Kort, kortlek, hand, regler, …

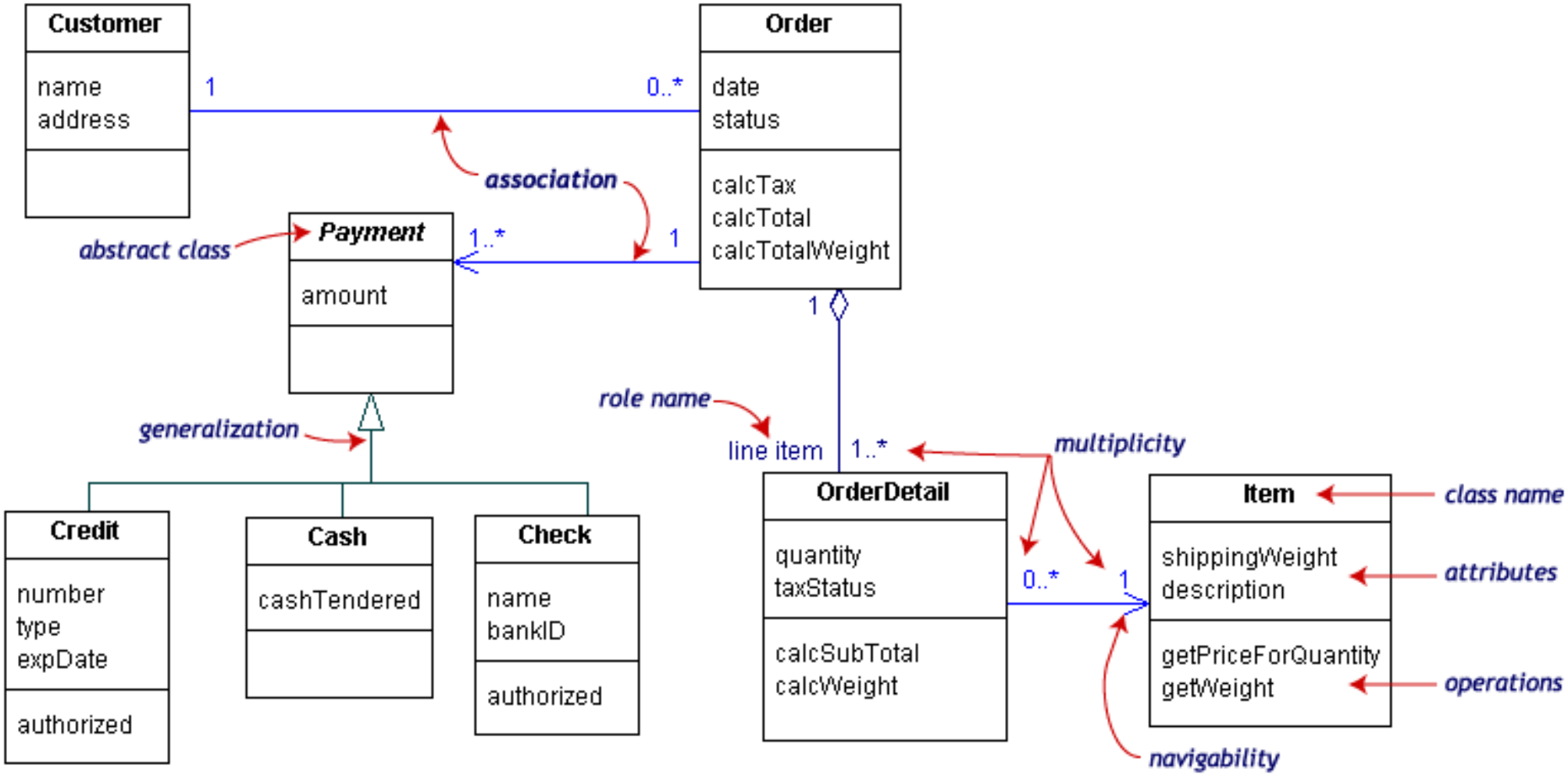
Programmeringsparadigmer



Java, C++, C#, Python, Ada, …

Source: http://atelier.inf.unisi.ch/

Modellera lösningen! Vad är detta?

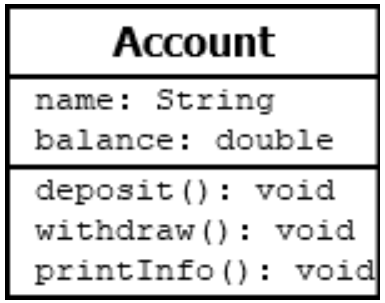


Objekt och klasser

- Objekt: En abstraktion av något i problemområdet, till exempel ett bankkonto eller ett spelkort
  - Har en egen identitet
  - Objekt har tillstånd, representerat via datamedlemmar
  - Operationer kan utföras på objekt: medlemsfunktioner, metoder
- Klass: En mall för objekt av samma typ
  - Vi kan skapa många, individuella objekt, utifrån denna mall
  - Objekt kallas även instans av en klass, ”instance of a class”
- Det närmaste vi kommer klass/objekt i C: en struct

En klass för ett bankkonto

- *Operationer, metoder*:
  - sätta in (deposit)
  - ta ut (withdraw)
  - skriva ut info
  - …
- Representation av *tillstånd*, datamedlemmar:
  - innehavarens namn
  - saldo (balance)
  - …



- *Kontrakt*:
  Syftar på villkor objekten måste uppfylla
  - Gränser/regler för data och …
  - … hur data ska kunna manipuleras
- Kontrakt i fallet bankkonto:
  - balance alltid >= 0
  - ett konto ska alltid ha en innehavare

En klass för bankkonton, första försöket

```
public class Account1 {

    // fields / data members
    double balance;
    String name;

    // member functions / methods
    void deposit(double amount) {
        if (amount >= 0) {
            balance += amount;
        }
    }

    void withdraw(double amount) {
        if (amount >= 0 && amount <= balance) {
            balance -= amount;
        }
    }
    …
```

```
    …
    void printInfo() {
        String info = String.format(
            "name: %s, balance: %.2f",
            name, balance);
        System.out.println(info);
    }
}
```

Spara källkoden i filen Account1.java

Skapa objekt (instans), första försöket

```
public class Account1Main {

    public static void main(String[] args) {

        Account1 acc = new Account1(); // create a new object (instance)

        acc.printInfo(); // name: null, balance 0.00

        acc.name = "Anders";

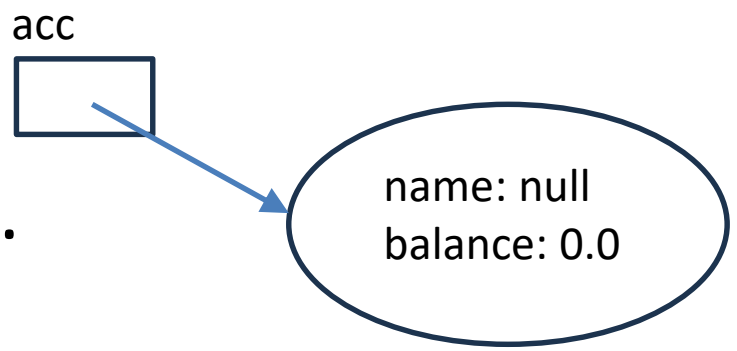
        acc.deposit(5000.0);
        acc.withdraw(2000.0);
        acc.printInfo(); // name: Anders, balance: 3000.00

        acc.balance = -1_000_000.0; // legal syntax, but breaks the contract(!)
        acc.printInfo();
    }
}
```

Spara källkoden i filen Account1Main.java

Skapa och accessa objekt (instans)

- Skapa objekt med operatorn ”new”
  - `Account acc = new Account();`
- Objektet hanteras via en referensvariabel …
  - … tänk en säker pekare, som endast kan peka på ett objekt av rätt typ …
  - … eller peka på inget alls (värdet null)
- Punkt-operatorn används vid
  - access till datamedlemmar, acc.name
  - anrop av metoder för ett givet objekt, acc.deposit(10.0);



Skapa objekt (instanser)

Källkod, t.ex. i main-metod

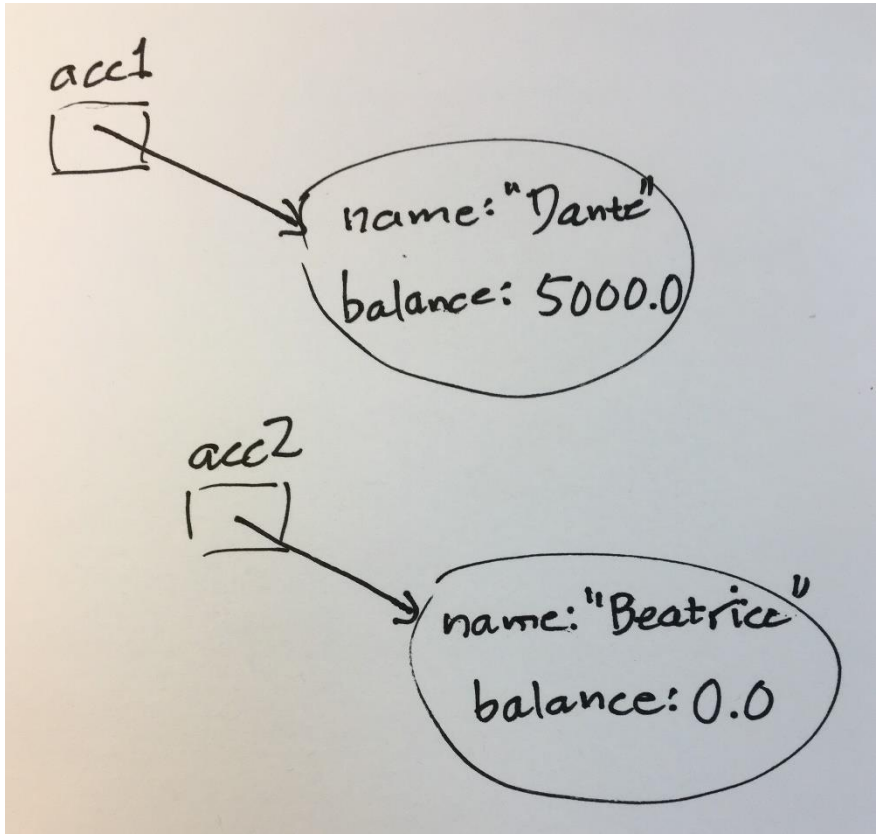
```
Account1 acc1 = new Account1();
acc1.name = ”Dante";
acc1.deposit(5000.0);

Account1 acc2 = new Account1();
acc2.name = ”Beatrice";
```

Skilj på

- referensvariabel, en ”säker pekare” och …
- … objektet (instansen)

I applikationens minne



Problem med Account1?

- Det går att bryta mot kontraktet (här: balance ≥ 0) eftersom datamedlemmen balance är direkt tillgänglig för andra klasser
- Vi vill kunna ”gömma” datamedlemmar för andra klasser …
  - … så att kontraktet inte bryts
  - Andra klasser ska inte veta något om den interna implementationen
- Datamedlemmarna initieras inte korrekt när objektet skapas
  - Vi får default-värden
  - 0/0.0 för int/double
  - null för objektreferenser

Inkapsling (encapsulation)

- Vi kan välja vilka datamedlemmar och metoder som ska vara synliga/tillgängliga för andra klasser …
- … och dölja andra datamedlemmar och metoder för andra klasser
  - private – endast synlig inom klassen
  - public – synlig för alla
- Poänger med inkapsling (några)
  - Vi kan garantera att objekt uppfyller ett kontrakt, dvs. att objektets tillstånd endast kan manipuleras på ett korrekt sätt …
  - … till exempel att balance alltid är ≥ 0
  - Vi kan dölja implementationen av vår klass för andra klasser …
  - … ”hide the implementation”, låt inte andra klasser ”se” det som de inte behöver känna till

Initiering av objekt, konstruktor

- En konstruktor är en metod som anropas då ett nytt objekt skapas
  - Account acc = new Account();
- Koden i konstruktorn ska initiera alla datamedlemmar korrekt
  - Datamedlemmar som inte initieras explicit får defaultvärden (0.0/false/null)
- En konstruktor ska ha
  - exakt samma namn som klassen, men …
  - … ingen returtyp (inte ens void)
- Om vi inte skriver en konstruktor i en klass genereras en parameterlös konstruktor
  - Sätter defaultvärden (0.0/false/null)
  - Undvik detta – skriv konstruktor(er) som initierar alla datamedlemmar korrekt

En klass för bankkonton – inkapsling, konstruktor

```
public class Account2 {

    // private fields
    private double balance;
    private final String name;

    // constructor, called when an object/instance is created
    public Account2(String name_) {
        name = name_;
        balance = 0.0;
    }

    // access methods
    public double getBalance() {
        return balance; // return a copy of the value
    }

    public String getName() { return name; }

    …
```

Inkapsling

- Datamedlemmarna name och balance är nu inte (direkt) åtkomliga i andra klasse?
- Men om vi vill avläsa värdet på balance eller name?
  - Vi skriver då publika så kallade access-metoder som returnerar *kopior* av privat data
  - getName, getBalance
- Notera också att konstruktorn nu garanterar att name alltid har ett värde …
  - … och att ”final name” innebär att name inte kan ändras efter att variabeln initierats

En klass för bankkonton, forts

```
    // mutator metods
    public void deposit(double amount) {
        if (amount >= 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount >= 0 && amount <= balance) {
            balance -= amount;
        }
    }

    // other methods
    @Override
    public String toString() {
        String info = String.format("name: %s, balance: %.2f", balance);
        return info;
    }
}
```

Metoden toString

- Alla java-klasser bör ha en egen version av metoden
  String toString()
- Metoden ska skapa en textsträng med information om objektets aktuella tillstånd …
  - dvs. datamedlemmarnas aktuella värden
  - Användbart till exempel för spårutskrifter vid felsökning av kod
- Lägg inte in utskrifter (System.out) eller inläsning (Scanner) i klasser som representerar data och logik
  - Bättre att returnera en textsträng, som kan presenteras på valfritt sätt, terminal, fil eller annat
- @Override? Har med arv att göra – förklaras senare i kursen …

Skapa objekt, försök 2

```
public class Account2Main {

    public static void main(String[] args) {

        Account2 acc = new Account2("Anders"); // initializing name and balance!
        System.out.println(acc.toString());    // name: Anders, balance: 0.00

        acc.deposit(5000.0);
        acc.withdraw(2000.0);
        System.out.println(acc.toString());

        acc.balance = 1_000_000.0; // private field – this is now a syntax error

        double copyOfBalance = acc.getBalance();
        System.out.println("Current balance is " + copyOfBalance);
    }
}
```

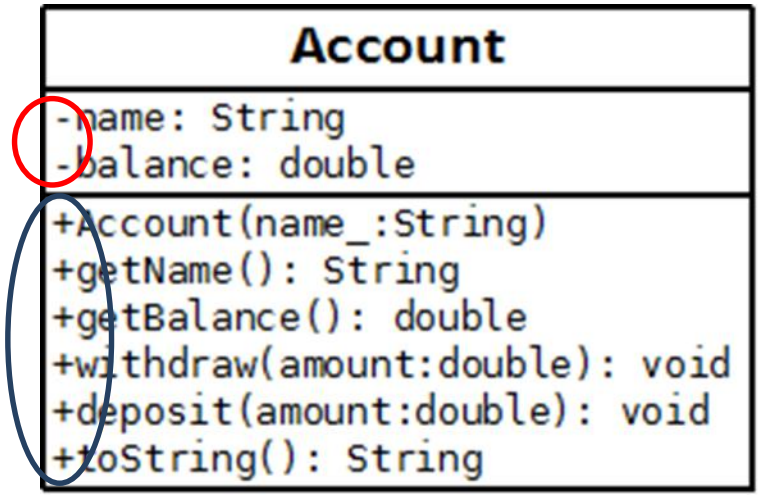
Synlighet, visibility, vid inkapsling i Java

Keyword/ UML-notation	Tillgänglig inom klassen?	Tillgänglig inom samma paket?	Tillgänglig i subklass?	Tillgänglig för alla andra klasser?
private -	Ja	Nej	Nej	Nej
“package private” ~	Ja	Ja	Nej	Nej
protected #	Ja	Ja	Ja	Nej
public +	Ja	Ja	Ja	Ja

Om vi inte anger någon synlighet alls får vi ”package private” (default)

Inkapsling och gränssnittet för en klass

- Med gränssnitt menas skiljelinjen mellan två system eller klasser …
  - … samt hur de kan kommunicera med varandra
  - Inte detsamma som användargränssnitt
- En klass publika gränssnitt är de metoder och data andra klasser ”ser”/har tillgång till …
  - … och definierar alltså vad andra klasser kan göra med objekten



Några problem med Account2

- Vi kanske vill kunna skapa objekt på olika sätt
  - Enbart med namn respektive med namn och startsaldo
- Kan vi slippa att hitta på nya namn för att skilja argument till konstruktor/metod från datamedlem?
  - name respektive name_
- Hur ska vi signalera om ett felaktigt uttagsförsök görs (mer än vad som finns på kontot)?
  - Notera att nuvarande kod ignorerar denna typ av uttagsförsök
  - Objektets saldo borde kontrolleras i main-metoden innan uttaget görs …
  - … men om den kontrollen inte görs?

En klass för bankkonton – flera konstruktorer

```
public class Account3 {

    // private fields
    private double balance;
    private final String name;

    // constructors
    public Account3(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }

    public Account3(String name) {
        this.name = name;
        this.balance = 0.0;
    }

    // access methods
    …
```

Namnkrock mellan lokala variabeln name (argumentet) och datamedlemmen name - this.name syftar alltid på datamedlemmen

Tre sätt att använda nyckelordet this

- Vi kan i en metod skilja på datamedlemmar och lokala variabler med samma namn
  - `this.balance = balance;`

- Vi kan i en konstruktor anropa en annan konstruktor med this
  - Nästa föreläsning …
- Vi kan referera till det objekt som exekverar metoden, t.ex. för att skicka det som argument till en annan metod
  - Nästa föreläsning …

Överlagrade metoder (overloading)

- Vi får ha flera konstruktorer eller metoder med samma namn i en klass …
- … förutsatt att antal argument och/eller typ på argumenten skiljer sig åt
  - Kompilatorn kan avgöra vilken metod- eller konstruktor-version som avses
  - Lämpligt när flera metoder logiskt sett utför samma operation, men med olika indata
-
```
class Time {
    …
    public void tic(int sec) { … }
    public void tic(int min, int sec) { … }
    public void tic(int min, double sec) { … }
}
```

En klass för bankkonton – signalera exekveringsfel

```
    // mutator metods
    public void withdraw(double amount) {
        if (amount < 0 || amount > balance) {
            // unexpected - terminate excecution of this method
            throw new IllegalArgumentException();
        }
        balance -= amount;
    }

    …

    // other methods
    @Override
    public String toString() {
        String info = "name: " + name + ", balance: ";
        info += String.format("%.2f", balance);
        return info;
    }
}
```

Skapa objekt, försök 3

```
public static void main(String[] args) {

    Account3 acc = new Account3(
        "Anders", 1000.0);

    System.out.println(acc.toString());

    acc.deposit(5000.0);
    acc.withdraw(2000.0);

    System.out.println(acc.toString());

    acc.withdraw(8000.0);

    System.out.println(acc.toString());
}
```

- När raden acc.withdraw(8000.0) exekveras *avbryts* exekveringen av applikationen
- "Stack trace":

```
Exception in thread "main"
java.lang.IllegalArgumentException
at Account3.withdraw(Account3.java:45)
at Account3Main.main(Account3Main.java:15)
```

Exceptions, undantag, för exekveringsfel

- När fel inträffar under exekvering, ex. felaktig indexering, avbryter JVM exekveringen av metoden och ”kastar” ett exception-objekt
  - Om detta inte hanteras i koden avbryts exekveringen av applikationen
- JVM kastar per automatik exempelvis
  - ArrayIndexOutofBoundsException vid felaktig indexering i array
  - AritmeticException vid division med noll
- Vi kan signalera fel genom att kasta exceptions i vår egen kod
  -
```
public void deposit(double amount) {
    if (amount < 0 && ..) throw new IllegalArgumentException();
    balance += amount;
}
```

Skapa objekt, försök 3

- Hur kan vi undvika att applikationen avbryts av det felaktiga anropet av withdraw?
- Gör, i anropande kod (main), en kontroll innan anropet
  - Använd metoden getBalance() för att kontrollera saldot innan anropet

```
public static void main(String[] args) {

    Account3 acc = new Account3(
        "Anders", 1000.0);
    …

    double amount = 8000.0;
    if(acc.getBalance() <= amount) {
        acc.withdraw(amount);
    }
    else {
        …
    }

    …
}
```

Klass/objekt, datamedlem, eller metod?

- Spelkort
- Dela ut kort
- Kortlek
- Valör för ett specifikt (spel-)kort
- Socket (en representation av en nätverksuppkoppling)
- Skicka data över nätverk
- Inköpslista
- Lägga till nytt inköp till en lista
- Transaktion

Klass/objekt, datamedlem, eller metod?

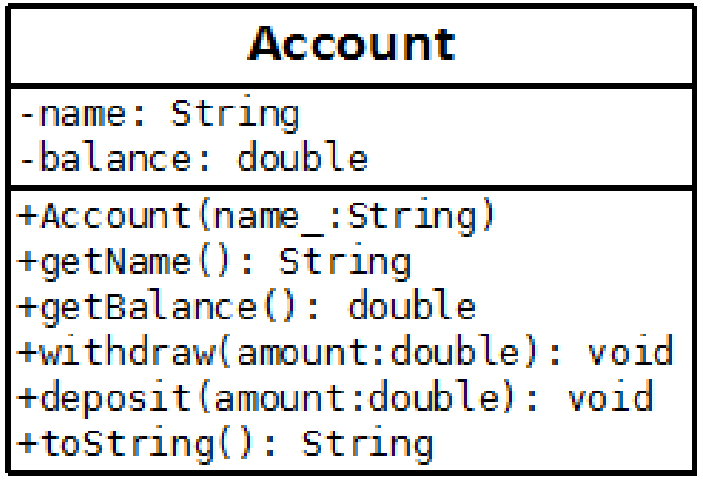
- Spelkort
- Dela ut kort
- Kortlek
- Valör för ett specifikt (spel-)kort
- Socket (en representation av en nätverksuppkoppling)
- Skicka data över nätverk
- Inköpslista
- Lägga till nytt inköp till en lista
- Transaktion?

Viktiga begrepp inom OOP (hittills)

1. Modellering
   i. Modellera objekt i verkligheten i programmet
   ii. Tillstånd (via datamedlemmar) + operationer (metoder)
2. Återanvändning
   i. Återanvändbara moduler = klasser
   ii. Förutsätter genomtänkt gränssnitt

Viktiga begrepp inom OOP

3. Inkapsling (encapsulation)
   i. Det som andra klasser inte bör/behöver känna till görs privat
   ii. Andra klasser har endast åtkomst till det publika, klassens gränssnitt (interface)
   iii. Allt som inte ingår i det publika gränssnittet är ”otillåtet”, och ”osynligt” för andra klasser



Överkurs: Publikt och privat i C?

- Ett bankkonto kan i C representeras med en ”struct”
- account.h – definierar gränssnittet för ”klassen”

```
struct account; // no information on the internal of this struct
typdef struct account Account;

void deposit(Account* a, unsigned int amount);

void withdraw(Account* a, int amount);

...
```

Överkurs: Publikt och privat i C?

- account.c
- Implementationen, inklusive datamedlemmar “göms” i denna fil

```
#include "account.h"

struct account {
        int balance;
        char name[80];
};

void withdraw(Account* a, int amount) {
        if(balance >= amount)  a->balance = a->balance - amount;
        else printf("Error, illegal withdrawal!");
}
…
```

Överkurs: Publikt och privat i C?

- main.c

```
#include "account.h"

int main() {

        Account acc;

        acc.saldo = 0;
        strcpy("Anders", acc.name);

        deposit(&acc, 1000);

        withdraw(&acc, 2000);

        acc.saldo = 1000000000; // saldo is not known here, illegal call
        ...
```